

시스템 구성방안 · 설계 사고 재구성

의료 AI 생성물 검증 게이트 시스템 구성방안

방법론 → 핵심 골격 → 실행 로드맵 통합본
백지에서 이 아키텍처에 "어떻게 도달하는가"

작성일: 2026-06-26

제1막 — 설계 방법론

이 시스템 설계의 핵심 난도는 "게이트를 만드는 것"이 아니다. 게이트는 쉽다. 진짜 어려운 4가지가 따로 있고, 설계 방법론은 이 4개를 어떤 순서로 공략하느냐로 정의된다.

설계를 지배하는 4개의 강제력 (forcing function)

#	강제력	이게 왜 설계를 지배하는가
F1	이질적 입력 → 균질 표현	AI는 HTML·TSX·FHIR JSON·번들을 뱉는다. 검사기는 단 하나의 표현(IR)만 알아야 한다. 안 풀리면 나머지 전부가 입력 형식에 오염.
F2	결정성 = 감사성	"같은 입력 → 같은 판정"이 깨지면 법적 증거능력이 사라진다. 비결정성을 구조적으로 봉인해야 한다.
F3	SaMD가 되지 않을 것	"우리가 의학적으로 판단했다"는 순간 의료기기 규제 사정권. 판정이 아니라 외부 권위 인용만 출력. 나중에 못 끼운다.
F4	'모름'을 끝까지 살릴 것	추론으로 배포를 막으면 (a) 오탐 신뢰붕괴, (b) SaMD 오인. unknown 이 게이트까지 살아남아야 한다.

방법론의 4단계

1단계 — 롱폴 리스크를 맨 앞에 놓는다

보통은 "빈 L3 규칙엔진"처럼 눈에 띄는 걸 헤드라인으로 삼는다. 틀렸다. 진짜 롱폴은 L1→L2 의미 추론 (Semantic Promoter)이다 — off-the-shelf 불가, day-1 필수, 모든 confidence 값의 출처. 첫 동작은 "가장 늦게 끝날 것"을 식별해 맨 앞에서 명세하는 것.

2단계 — '되돌릴 수 없는 결정'만 1일차에 동결한다

- 비가역(now): 스키마·인터페이스·불변식 — 나중에 끼우면 전면 재작성. → 1일차 동결
- 가역(later): 저장소·서버·규·격리 실물 — 나중에 국소 추가. → 2~3단계로 미룸

판별 기준 한 줄: "이걸 미루면 나중에 인터페이스를 바꿔야 하나?" 예면 동결, 아니면 미룬다. 이게 "스키마는 미래형, 코드는 현재형" 규율의 정체다.

3단계 — 제약을 문장이 아니라 타입/구조로 강제한다

"규칙은 순수 함수여야 한다"를 문서에 쓰면 안 지켜진다. `Rule.evaluate(ir): Violation[]` 시그니처에서 I/O를 불가능하게 만들면 지켜진다. F3은 `Violation.authority` 를 필수 필드로 만들어 스키마 검증이 거부하게 한다.

4단계 — 적대적 검토로 횡단 공백을 사냥한다

세 개의 독립 렌즈로 같은 설계를 공격한다 — ①실현가능성, ②기술 견고성, ③전략 해자. 각 렌즈가 다른 사각을 집는다. 단독 검토로는 안 나온다.

제2막 — 핵심 구조 골격

방법론을 적용하면 시스템은 6개의 본질 구성요소로 수렴한다. 나머지는 전부 이 6개의 부수물이다.

```
[1 Capture]→[2 IR]←(만든다)[3 Lift·Promote·Bind]
      ↓ (읽기 전용)
[4 Rule Engine]→[5 Gate]→[6 Audit/Store]
```

#	요소	한 줄 정의	의존	본질/부수
1	Capture 경계	이질적 아티팩트를 결정적 스냅샷으로 균질화	외부 입력	본질 (F1-F2)
2	IR (다층)	모두가 말하는 단일 계약. 구조/의미/도메인 3층	없음	본질 (척추)
3	Lift→Promote→Bind	구조→의미로 기어오르며 confidence/status 산출	IR, facts	본질 (★리스크)
4	규칙 엔진	IR을 읽어 authority 붙은 위반 방출. I/O 금지	IR만	본질 (F3)
5	심각도·게이트	block/warn을 결정하는 단일 길목	위반·정책	본질 (F4)
6	감사·룰 저장소	불변 해시체인 기록 + 버전드 룰	게이트 결과	본질+부수

골격을 떠받치는 3개의 "강제 경계"

경계 ① — 단방향 흐름 + IR 단일 언어

어댑터는 규칙을 모르고, 규칙은 어댑터를 모른다. 둘 다 IR만 안다. 데이터는 좌→오로만 흐르고 IR은 생성 후 read-only. → 어떤 어댑터/규칙을 추가해도 코어 변경 0.

경계 ② — 결정적 nodeId가 유일한 조인 키

`nodeId = layer:sha1(tenantId || stablePath)`. L1↔L2↔L3 결합, 시나리오 dedup, 골든 스냅샷의 유일한 부하 지지점. 비결정적이면 F2(감사성)가 통째로 무너진다. 골격에서 가장 과소평가된 핵심.

경계 ③ — '모름'과 '권위'의 양끝 타입 강제

입력단에 `status: known|unknown|ambiguous` (F4)와 `Measured<T>` ('미측정'≠'통과'), 출력단에 `authority` 필수(F3). 이 두 타입이 파이프라인 양 끝을 봉인한다.

골격의 단일 수렴점 — 게이트 불변식

위 모든 게 결국 **한 함수**로 모인다. 차단(block)이 성립하려면:

```
status==='known' ^ confidence ≥ floor ^ rolloutState==='block'
```

미상·저신뢰·미승급 중 **하나라도** 있으면 절대 block 불가. F3·F4가 이 한 지점에서 강제된다. 골격을 한 문장으로: "관측된 사실만 강하게 막고, 추론은 경고로 강등한다."

제3막 — 실행 로드맵

골격을 코드로 옮길 때의 구성. 핵심 규율: **패키지 경계 = 아키텍처 경계**.

패키지 분할 (pnpm 모노레포)

```
@sb/ir-schema      ← 키스톤. 모두가 import. 경계 드리프트 컴파일타임 차단
├ @sb/adapters/*    (Capture만 - dom-snapshot, fhir, tsx-static)
├ @sb/lift           (Capture→L1 균질화 + 결정적 nodeId)
├ @sb/normalize      (axe-core→facts[] 매핑)
├ @sb/promoter       (L1→L2 의미 추론 - ★리스크 집중)
├ @sb/contract-binder (L2+카탈로그→binding.contract)
├ @sb/rules/*        (순수 함수 룰)
├ @sb/engine         (평가·dedup·effectiveSeverity)
├ @sb/report         (리포트·SARIF·exit-code)
├ @sb/cli            (얇은 클라이언트)
└ @sb/eval-server    (2단계 - IP 격리)
```

기술 스택 — 선택의 이유만

영역	선택	비가역적 이유
언어	TS + Node	AST·DOM 생태계가 Node 집중. IR을 단일 패키지 타입으로 전 모듈 공유
런타임 DOM	Playwright	마운트 한 번으로 대비비·포커스순서·a11y트리·axe를 공짜로 동시 확보 (F1 우회)
IR 검증	Zod	외부 아티팩트라 런타임 검증 필수. authority 부재 거부를 스키마단에서 강제 (F3)
접근성	axe-core	검증된 룰 재사용. 1단계 5규칙을 빠르게

무재작업 빌드 순서 (이 순서가 핵심)

① @sb/ir-schema 동결

← status·nodeId·contract 슬롯. 모든 것의 전제

② dom-snapshot Capture + Lift

← 결정적 nodeId 검증(골든 픽처)

③ NORMALIZE (axe→facts)

④ Semantic Promoter

← ★1순위 리스크. 가장 먼저·집중

⑤ ContractBinder (빈 카탈로그 통과)

⑥ 규칙 5종 + 엔진 + effectiveSeverity(게이트 불변식)

⑦ DEDUP

⑧ CLI 리포트 + fail-closed 검증 + exit-code 계약

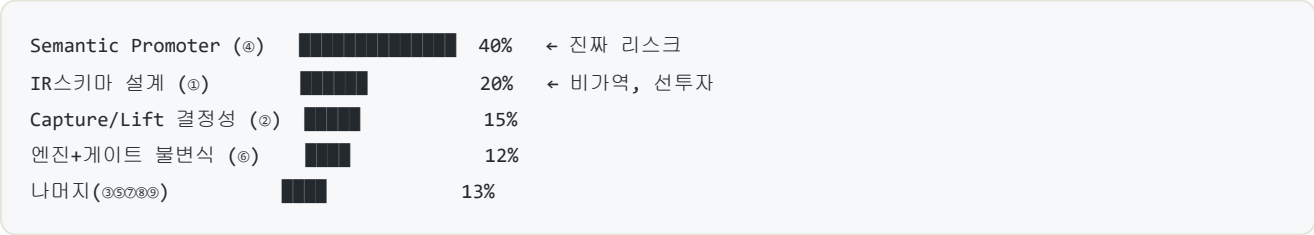
⑨ 감사 레코드 (prevHash 체인 형성)

순서의 논리: ①을 못 박지 않고 ②~⑨를 짜면 전부 재작업. ④를 뒤로 미루면 "리스크를 안 본 채 출하 직전 폭발". 키스톤 먼저, 리스크 그다음.

단계별 동결 vs 유보

	1단계 (0~3M·데모)	2단계 (3~9M·CI게이트)	3단계 (9M~해자)
동결 (now)	IR스키마, nodeId규약, exit-code, prevHash 규약, tenant키	(1단계 동결분 소비)	—
유보 (later)	게이트·서버·DB·격리	Postgres·서버평가 ·SARIF·PHI레딕션	L3 의료주식·계약정합성·IP해자

노력을 어디에 쏟나 (자원 배분)



직관과 반대다. "규칙엔진"이 제일 화려해 보이지만 거기 쏟으면 안 된다. Promoter 휴리스틱의 실증 정확도가 전체 신뢰도를 좌우하므로 거기에 골든 데이터셋과 시간을 몰아야 한다.

한 줄 요약 — 비가역 결정(스키마·불변식)을 1일차에 선투자해 미래 재작업을 막고, 가역 인프라는 2단계로 미뤄 출하 속도를 산다. 노력은 가장 늦게 끝날 Promoter에 집중한다.